

CUDA PROGRAMMING FOR NVIDIA H100S

cp.async.bulk

Prateek Shukla · 简体中文版

cp.async.bulk 操作

cp.async.bulk 是一组 PTX 指令，用于在 Hopper H100 GPU 上进行硬件加速的异步 bulk 内存传输。

这些操作被下放给专用硬件，独立于 SM 的计算流水线执行，使计算可以在数据传输于后台进行的同时继续推进。

cp.async.bulk 能够高效处理大型多维 tensor 传输，覆盖从 1D 到 5D、包含数百到数千字节的 tensor。

cp.async.bulk 操作需要屏障对象来做协调，确保异步传输与计算操作之间有正确的顺序。

cp.async.bulk 与 cp.async (ampere)

ampere 上的 cp.async 使用 Load/Store Unit (LSU)。它之所以是“异步”的，是因为线程发射指令后就继续前进，但线程仍然要为每 16 字节数据计算地址并发射命令。hopper 上的 cp.async.bulk 使用张量内存加速器 (TMA)。单条线程发射一条指令即可拷贝整个 tile，TMA 在后台处理所有地址计算、循环展开和搬运。

更多关键区别

`cp.async (ampere)` : 要拷贝一个 4KB 的数据 tile , warp 中的每条线程都必须循环并发射多条 `cp.async` 指令。这会烧掉寄存器周期和指令缓存。使用 `cp.async.commit_group` 和 `wait_group`。`cp.async.bulk (hopper)` : 单条线程就可以为整个 block 发起传输。warp 中其余 31 条线程 (或 block 中的 127 条) 实际上与内存拷贝的发起毫无关系。配合 `mbarrier` , TMA 硬件会在字节到达时自动更新屏障的“事务计数”。

cp.async.bulk 的布局

1. tensor 布局

这是当你持有 TMA 描述符 (Tensor Map) 并想拷贝特定多维 tile 时使用的布局。

```
cp.async.bulk.tensor.ndim.dst.src.barrier_type{.cache_hint}{.multicast}
```

```
dst_addr, tensor_map, coordinate_array, mbarrier_addr;
```

2. 原始布局 (线性)

这是用于简单、连续字节拷贝的布局。cp.async.bulk.dst.src.barrier_type{.cache_hint}

```
dst_addr, src_addr, size, mbarrier_addr;
```

cp.async.tensor

在指令中加入 `.tensor` 表示该操作是 tensor 感知的，也就是说指令作用于多维 tensor 数据而不只是扁平数组。

这还打开了可以在指令中设置的许多其他非常重要的选项。这也是能够使用 `cuTensorMap` 的原因。

cp.async.bulk.tensor.{1d,2d,3d,4d,5d}

1d/2d/3d 告诉硬件：它需要从你这里读取多少个数字（索引）才能定位该 tile。

所以基本上，一旦拿到在 tensor 中定位 tile 所需的 n 个索引，我们就可以按照 cuTensorMap 给出的信息抓取该 tile。

这里的 Nd 表示 tensor 有 N 个维度。

我们可以使用最多 5 个维度的 tensor。

源与目标的状态空间

In `cp.async.bulk.tensor.{dim}.{space1}.{space2}`

`space` 表示源 tensor 的状态空间。

`space2` 表示目标 tensor 的状态空间。

它们可以是我们在前面讨论过的任意一种：`global`、`shared::cta`、`shared::cluster` 等。

加载模式 `{.tile, .im2col, .im2col::w ....:}`

这个修饰符至关重要，因为它告诉 TMA 硬件如何解释你提供的坐标，以及如何在传输过程中就地变换数据。

`.tile` —— TMA 使用 `tensorCoords` 中提供的坐标计算基地址，并遵循 `tensorMap` 中定义的步长，抓取一个致密、连续的多维数据盒（一个 tile）。

im2col

它在抓取过程中执行硬件加速的 im2col 变换。过去你必须编写一个 kernel，把像素从图像布局 (N,C,H,W) 拷贝成列布局（矩阵）。这浪费内存带宽和寄存器空间。

现在你只需给出卷积窗口左上角的坐标，TMA 就会抓取滤波器所需的像素，把它们展开，并像矩阵的一列一样放入 L2，从而加速该操作。

完成机制

`cp.async.bulk.tensor.{}.{}.{}.completion_mechanism`

我们有两种完成机制

- `mbarrier::complete_tx::bytes`
- `.bulk_group`

mbarrier*:complete_tx*:bytes

cp.async.bulk 调用接受 mbarrier 句柄（很可能放在 [mbar] 参数中）以及完成机制说明。硬件会跟踪这次操作，并在数据搬运时自动更新屏障的 tx-count。

当 tx-count 归零时，屏障 phase 翻转，等待的线程被释放。

你需要一个指向 mbarrier 的指针以及拷贝操作的大小，作为该操作的操作数。

bulk_group

`bulk_group` 是比 `mbarrier` 简单得多、也更轻量的替代方案。

你不必跟踪 `tx_count`，而是用 `bulk_group` 发射一批拷贝操作，然后用 `commit_group` 把它们打包成组，再用 `wait_group` 等待，直到最近的 `bulk` 异步组中只剩不超过 `N` 个仍处于 `pending` 状态。

L2 缓存提示

```
cp.async.bulk.tensor.1d.shared*:cta.global.mbarrier*:complete_tx*:bytes.L2*:cache_hint
```

在异步拷贝中，如果任由数据灌满 L2 缓存，它会挤掉你可能正在反复使用的其他重要缓存数据。为了防止一次性数据占用 L2 缓存中的有用空间，我们使用 L2 提示。加载和存储有不同的实现方式。evict_first 是告诉硬件立即丢弃数据以节省缓存空间的方式。evict_last 是告诉硬件把数据标记为持久并尽可能长时间保留的方式。evict_normal 是默认行为。

L2 提示加载 (**global -> shared/dsmem**)

在 global -> shared 传输过程中，数据按照写回 (write back) 缓存策略被加载并缓存进 L2。

对于被复用的权重/数据 (最佳) ：使用 `evict_last`。这会把数据标记为“持久”，告诉 L2 缓存尽可能长时间保留它。

对于流式/单次数据：使用 `evict_first`。因为你本质上是用完一次就丢掉。

L2 提示存储 (**shared -> global**)

把结果写回全局内存时，你通常不需要立即再次读取它们。

对于最终输出（最佳）：使用 `evict_first`。你是在把数据写入 HBM，本 SM 很可能不会再把它读回来。

这能把缓存污染降到最低。数据会经过 L2（以完成写入），但立即被标记为第一驱逐候选，从而为你的输入保持缓存干净。

创建缓存策略描述符

PTX 中的 `createpolicy` 指令创建一个 64 位缓存策略描述符，为特定的内存访问模式编码驱逐优先级。它的工作方式如下：

缓存策略描述符是一个决定缓存策略的 64 位对象。`createpolicy.fractional.L2*:evict_last.b64 policy_reg, 1.0;`

`createpolicy.fractional.L2*:evict_first.b64 policy_reg, 1.0;`

`dest` 是将持有缓存策略描述符的 64 位寄存器。

`fraction(1.0)` 决定该策略应用于多大比例的数据。

拷贝非结构化数据与结构化（**tensor**）数据

`cp.async.bulk`（非结构化）是硬件加速的 `memcpy`。它把内存当作线性字节流，并不“知道”你的数据是矩阵、3D 体积还是分块 `tensor`。

我们使用 `cp.async.bulk.shared/cp.async.bulk.global` 来拷贝非结构化数据。

`cp.async.bulk.tensor`（结构化）是智能的、基于描述符的拷贝。它依赖于在主机上创建的 `CUTensorMap` 对象（不透明句柄）。硬件“理解”你数据的维度、步长和边界。

我们使用 `cp.async.bulk.tensor` 拷贝结构化 `tensor`，`cuTensorMap` 在这里登场。

CUDA PROGRAMMING FOR NVIDIA H100S

非结构化拷贝

Prateek Shukla · 简体中文版

操作数：

`dstmem` —— 共享内存中的目标地址

`srcMem` —— 共享内存中的源

`size` —— 以字节计的传输数据大小

`mbar` —— 指向你正在使用的 `mbarrier` 对象的指针

`cache_policy` —— 指向策略描述符的 64 位指针

操作数：

Dst : global 中的目标地址

Src : 拷贝的来源

Size : 事务大小

Cache_policy : 缓存策略的描述符

Mask : 用于掩码写入，指定要写入目标的哪些字节

dst —— dsmem 中的目标地址

src —— 共享内存中的源地址；size —— 拷贝大小；completion_mechanism —— mbarrier。这里的要点是：你不是在把数据传输到分配给其他某个 block 的共享内存，而是在把数据移动到分布式共享内存（distributed shared memory）中由所有 block 池化的某个位置。

multicast 的由来

计算数据比搬运数据快。尤其是从 HBM 搬运数据，非常耗时耗能。计算本身可以快得多。

许多深度学习应用都使用 GEMM，而在 GEMM 中，许多不同的 block 需要读取矩阵 A 的同一块，去乘以它们各自的那块矩阵 B。比如 SM0、SM1、SM2、SM3 都需要“Tile A0”。

在上一代架构中，如果 4 个 SM 都需要同一份数据（某个神经网络层的权重矩阵），这 4 个 SM 就必须各自向全局内存请求这份数据（代价非常高）。

为什么不从 HBM 只取一次数据，并在它流经线路时同时拷贝给全部 4 个 SM？

操作分解

TMA 从全局内存 (srcMem) 读取 size 字节到 L2 缓存。缓存提示告诉 L2 把这一行保持住，为其他波 (wave) 的后续访问优化带宽。

L2 缓存控制器把数据读取一次，并通过集群 crossbar 广播，同时瞄准 multicast 掩码中定义的每个 block 的 SMEM bank。

L2 缓存控制器把数据读取一次，并通过集群 crossbar 广播，同时瞄准 multicast 掩码中定义的每个 block 的 SMEM bank。

完成时，TMA 使用 mbar 指针 (同样以 multicast 方式编码)，同时给每个参与 block 中 mbarrier 的事务计数原子地加上 size 字节。

由单条 leader 线程发射这条非阻塞指令。TMA 硬件独立管理整个“抓取-广播-通知”流水线，让所有 block 中的线程可以在等待期间继续计算或休眠。

mbarrier 与 multicast

这里很容易搬起石头砸自己的脚，因为关于在集群中为特定线程块使用 mbarrier，有一些必须理解的东西。

mbarrier 对象会在每个参与 CTA 的共享内存中以相同的相对内存偏移复制一份。当生产者发射 TMA 指令时，硬件把数据广播到 ctaMask 中的所有 CTA，并自动给每个目标 CTA 中该特定地址上的 mbarrier 发信号。

硬件从指令的掩码中立刻知道组成员有哪些。接收 CTA 不需要通过“arrive”来组成组；它们只需在自己本地的 mbarrier 实例上等待，就能知道有效数据何时落定。

集群中所有 **block** 的第一条线程（或任意单条线程）调用 **expect_tx**

并传入它们期望的数据量，而且都指向自己共享内存中的 mbarrier。cp.async 跟踪屏障的方式是使用偏移量来定位每个 block 的 mbarrier。

整个 block 中由单条线程调用带 multicast 的 cp.async.bulk，并附带该集群中所有 block 的掩码，它指向自己的屏障。

如果你想把数据传输到 block 0 和 3 而不是 block 1 和 2，那么你绝不能让 block 1 和 2 调用 mbarrier，也不要把它们放进掩码。

所有 block 中的所有消费者 warp 都在自己本地的屏障上自旋等待，即 mbarrier.try_wait。

对于被掩码选中的 block，到达计数由 TMA 自己管理。TMA 会自动向指定偏移处的 mbarrier 发出“远程到达”（remote arrival）信号。

一个小提示

使用动态共享内存时，你需要手动管理共享内存布局。

不要这样做——`extern *_shared* _char smem[];` 然后再添加 `mbarrier`

改用这样——`uint64_t* bar_ptr = reinterpret_cast<uint64_t*>(smem); int tma_alignment = 128;`

`int data_offset = (sizeof(uint64_t) + tma_alignment - 1) & ~(tma_alignment - 1);`

`half* tile_ptr = reinterpret_cast<half*>(smem + data_offset);`

所有参与的 **CTA**（比如说，同一集群内运行在 **16** 个不同 **SM** 上的 **16** 个 **CTA**）必须首先把自己确立为一个 **multicast**

组。每个 CTA 在共享内存中分配相同的 mbarrier 对象，用到达计数调用 `mbarrier.init.shared.b64`，然后在该 mbarrier 上执行 `arrive()`。这次到达不只是同步——它是硬件注册。内存子系统由此知道这 16 个 CTA 组成了一个将接收相同数据的逻辑组。

multicast 组中的每个 CTA 都创建相同的 `CUtensorMap` 描述符。在主机端，你调用 `make_tma_copy(SM90_TMA_LOAD_MULTICAST{}, gmem_tensor, smem_layout, cluster_size)`，它会编码 tensor 几何形状、数据类型、swizzle 模式，以及至关重要的集群维度。这个描述符被传给 kernel（标记为 `__grid_constant__`），告诉 TMA 要抓取哪个全局内存区域、以及放到每个 CTA 共享内存的哪个位置。全部 16 个 CTA 必须使用这个完全相同的描述符——任何偏差都会破坏“它们想要相同数据”这一契约。

每个 CTA（通常由每 CTA 单条被选举出的线程）发射 TMA multicast 指令：

`cp.async.bulk.tensor.shared.cluster.global.mbarrier.multicast`。注意 `.cluster` 作用域和 `.multicast` 限定符——它们表达的是硬件意图。该指令接受共享内存目标地址、`tensorMap` 指针、tensor 坐标、mbarrier 指针，以及至关重要的 `ctaMask` 参数。`ctaMask` 是 16 位掩码（集群大小为 16 时），其中第 *i* 位表示 CTA *i* 是否参与——例如 `0xFFFF` 表示全部 16 个 CTA 都接收数据。

所有参与的 CTA（比如说，同一集群内运行在 16 个不同 SM 上的 16 个 CTA）必须首先把自己确立为一个 multicast(续)

这就是 L2 缓存层面发生魔法的地方。L2 缓存控制器收到 16 个看似独立、实则都指向同一个 tensor tile 的请求，它们都带着相同的 mbarrier 组 ID。硬件识别出这种模式，并把其中一个请求提升为 leader。这个 leader 请求触发对 HBM 的一次读取（比如说 1MB 的权重数据）。当数据流入 L2 缓存时，缓存控制器并不只把它发给一个 SM——而是把数据 multicast（同时转发）到全部 16 个参与 SM 的 L1 缓存，并直接进入它们的共享内存区域。你只付出 1MB 的 HBM 带宽，却在各 SM 之间送达了 16MB 的数据。

TMA 硬件还会在数据到达时自动递减每个 CTA 的 mbarrier 上的事务字节数（tx-count），跟踪完成进度。

发射 TMA multicast 之后，每个 CTA 执行 `wait_barrier(tma_load_mbar, phase)` 或 PTX 中等价的 `mbarrier.try_wait`。这会阻塞，直到 mbarrier 的事务计数归零——意味着全部预期字节都已送达该 CTA 的共享内存。一旦所有 CTA 都通过这个屏障，它们就得到保证：共享内存已被数据填充，计算可以开始。

Dst —— 目标地址

Src —— 全局内存指针；Size —— 操作的大小；Mbar —— 指向 mbarrier 的指针；Ctmask —— 用于 multicast 的 16 位掩码；Cache-policy —— 缓存策略。

CUDA PROGRAMMING FOR NVIDIA H100S

结构化拷贝

Prateek Shukla · 简体中文版

把数据从 **global** 拷贝到 **cta**

操作数 -

dstMem —— 指向共享内存位置的指针

tensorMap, tensorCoord —— tensorMap 的地址、box 坐标数组

srcMem —— 指向全局内存地址的指针

cache-policy —— 指向缓存描述符的指针

操作数 -

dstMem —— 指向 dsmem 的指针；tensorMap, tensorCoord —— tensorMap 的地址、告知坐标的 1D 向量；mbar —— 指向 mbarrier 对象的指针；ctaMask —— 用于选择要拷贝到哪些 block 的掩码；cache-policy —— 指向缓存描述符的指针。

使用 **bulk_group** 的 TMA 存储

tensorMap, tensorCoords —— 指向 cuTensorMap 的 64 位指针、box 坐标数组

srcMem —— 数据来源的内存位置

Cache-policy —— 指向缓存策略描述符的指针

cp.async.bulk.prefetch

我们可以把数据预取到 L2 缓存以获得更低延迟。

我们使用 `cp.async.bulk.prefetch.tensor` 来做到这一点。

几个要点

`cp.async.bulk.prefetch` 是一个性能提示：如果你发射了 `cp.async.bulk.prefetch`，然后立即发射 `cp.async.bulk.tensor`，而数据尚未被缓存，那么设备照样会从 HBM 读取数据。

即使主 `cp.async` 操作和预取操作使用同一个 `tensorMap`，L2 提升和 `swizzle` 对数据在 L2 中被缓存的方式也没有影响。

cp.reduce.async

它把整个数据 tile 的原子累加从 SM 下放给 TMA。

这正是 TMA 用该指令所做的工作。下面是两个版本。

```
cp.reduce.async.bulk.dst.src.completion_mechanism{.level::cache_hint}.redOp.type [dstMem],  
[srcMem], size{, cache-policy}
```

```
cp.reduce.async.bulk.tensor.dim.dst.src.redOp{.load_mode}.completion_mechanism{.level::cache_hint} [tensorMap, tensorCoords], [srcMem] {,cache-policy}
```

允许的 **redOp** 与数据类型

.add .f16

.min/.max .bf16

b32 .inc/.dec u32 .and s32 .or b64 .xor u64

s64

f32

f64